

NLP Course

324827328

Part A.1

Why does the program generate so many long sentences? Specifically, what grammar rule is responsible for that and why? What is special about this rule? discuss.

The only unbounded recursion in the base grammar is $\text{NP} \rightarrow \text{NP PP}$. It keeps expanding NPs with more PPs (“the sandwich on the desk in the room ...”) and drives long sentences. Rules like $\text{VP} \rightarrow \text{Verb NP}$ are not recursive by themselves; length only blows up when $\text{NP} \rightarrow \text{NP PP}$ fires repeatedly.

The grammar allows multiple adjectives, as in: “the fine perplexed pickle”. Why do the generated sentences show this so rarely? discuss.

To generate an adjective, we must choose the following rule to continue from noun “1 Noun Adj Noun”, but this rule is equally distributed with the following rules:

1. 1 Noun Adj Noun (the rule we want)
2. 1 Noun president
3. 1 Noun sandwich
4. 1 Noun pickle
5. 1 Noun chief of staff
6. 1 Noun floor

Which means we only have a $\frac{1}{6}$ probability of choosing the rule “1 Noun Adj Noun”. As a result, we don't have much adjectives.

Which numbers must you modify to fix the problems in (1) and (2), making the sentences shorter and the adjectives more frequent?

To make the sentences shorter we mainly lower the recursive $\text{NP} \rightarrow \text{NP PP}$ weight (to 0.1) and boost the base NP rule ($\text{NP} \rightarrow \text{Det Noun}$ to 10), so

expansions terminate instead of chaining PPs. Raising terminal-preterminal weights (e.g., common Verbs/Det/Nouns) also helps end derivations faster but the key change is dampening the recursive PP attachment.

To make adjectives more likely we raise the adjectival recursion weight $\text{Noun} \rightarrow \text{Adj Noun}$ (to 20) so modifiers are favored over bare noun choices; adjectival lexicon weights stay balanced.

What other numeric adjustments can you make to the grammar in order to favor a set of more natural sentences? Experiment and discuss.

We can do the following:

1. We can increase the weights for "the" and "a" dramatically compared to "every.". "The" is the most common word in the English language, making it far more "natural" to see it often in generated sentences than "every.". Overall we change the following rules:
 - (a) 50 Det the
 - (b) 25 Det a
 - (c) 5 Det every
2. We can try adjusting sentence structures (ROOT). simple declarative statements (S .) are much more common than exclamations or complex questions. We make the following changes:
 - (a) 10 ROOT S .
 - (b) 1 ROOT S !
 - (c) 0.5 ROOT is it true that S ?
3. We can try favoring common Verbs. We make the following changes:
 - (a) 8 Verb ate
 - (b) 8 Verb wanted
 - (c) 5 Verb kissed
 - (d) 2 Verb understood
 - (e) 0.1 Verb pickled

Part A.2

Modifications to the Grammar

I modified the grammar to support the required sentences (1)-(10) by adding new vocabulary and rules. Key changes include:

- Separated verbs into subcategories: VerbT (transitive), VerbI (intransitive), VerbThat (sentential complement), Copula (is), VerbPerplex (specific for "it perplexed...").
- Added coordination rules for NPs ($\text{NP} \rightarrow \text{NP Conj NP}$) and VPs ($\text{VP} \rightarrow \text{VP Conj VP}$).
- Added a specific rule for transitive verb coordination ($\text{VerbT} \rightarrow \text{VerbT Conj VerbT}$) to handle "wanted and ate a sandwich".
- Added support for "It-cleft" structures like "it perplexed NP that S".
- Added recursive adjective modification ($\text{Adj} \rightarrow \text{Adv Adj}$).

Interaction between Conjunctions and Copula/Progressive

Handling sentences like (2) "Sally and the president wanted and ate a sandwich" and (8)/(9) "Sally is lazy" / "Sally is eating a sandwich" can be problematic if categories are not strictly defined. For example, if we simply classified "wanted", "ate", and "is" all as Verb, and allowed $\text{Verb} \rightarrow \text{Verb and Verb}$, we might generate ungrammatical sentences like:

* Sally wanted and is lazy.

Here, "wanted" requires an NP object, while "is" in this context takes an Adjective. Coordinating them creates a mismatch where "lazy" cannot serve as the object for "wanted". Similarly, "Sally wanted and sighed a sandwich" would be generated if we coordinated transitive and intransitive verbs. To avoid this, my grammar distinguishes between VerbT, VerbI, and Copula, and only allows coordination between compatible types (e.g., VerbT with VerbT).

Part A.4

Extensions to the Grammar

I extended the grammar to handle two additional linguistic phenomena:

1. (e) **Singular vs. plural agreement**
2. (b) **Yes-no questions**

Singular vs. Plural Agreement

To handle agreement, I split the relevant categories into Singular (Sg) and Plural (Pl) variants:

- **Nouns and NPs:** Split into NounSg, NounPl, NPSg, and NPPl.
- **Verbs and VPs:** Split into VerbTSg/Pl, VerbISg/Pl, CopulaSg/Pl, etc.

- **Sentence Structure:** The S rule enforces agreement:

- S → NPSg VPSg
- S → NPP1 VPP1

This ensures that "the president eats" (Sg-Sg) and "the presidents eat" (Pl-Pl) are generated, while "*the president eat" is blocked.

Yes-No Questions

I added support for Yes-No questions using two strategies:

1. **Auxiliary Support:** do/does/did/will plus an uninflected VPBase (ROOT → Aux NP VPBase ?). VPBase excludes copular be, so we avoid ungrammatical forms like "does the president be...".
2. **Copula Inversion:** Dedicated rules invert the copula directly (ROOT → CopulaSg/Pl NP Adj/NP/VPing/PP ?), yielding "Is the president lazy?" / "Are the presidents eating?" without do-support.

For progressives, I split **VerbIng** into transitive and intransitive forms so **VPing** requires an object only when the verb subcategorizes for one, preventing frames like "was the sandwich eating ?".